Performance Profiling

measure where the time goes, don't guess

Watch the 4-min video

In one line: profile before optimising: sampling profilers like py-spy attach to a running process with low overhead and produce flame graphs where width is time; fix the widest frame, measure again; cProfile is exact but heavy, Scalene and memray add memory views, and CPU profiles don't show waiting, which needs tracing.

1 A day spent optimising the wrong thing

Your Python service runs at 100% CPU and p99 latency is climbing. You think it's the database code, so you spend a day optimising it. Nothing changes. **Never guess where time goes; measure it.** That's profiling.

2 Two kinds of profiler

Deterministic profilers, like `cProfile`, hook every function call and record exact counts and times, but add a lot of overhead and distort fast functions. **Sampling** profilers, like `py-spy`, look at the stack many times per second and count where the program is. Low overhead, and safe for production.



Figure 1. Exact but heavy, or statistical and light.

3 Reading a flame graph

Each box is a function, stacked on top of its caller. The **width** of a box is its share of samples, so wide boxes are where the time goes. Read it bottom up and look for the widest plateaus. Left to right is *not* time; it's just sorted alphabetically. A narrow tower is deep, but not expensive.

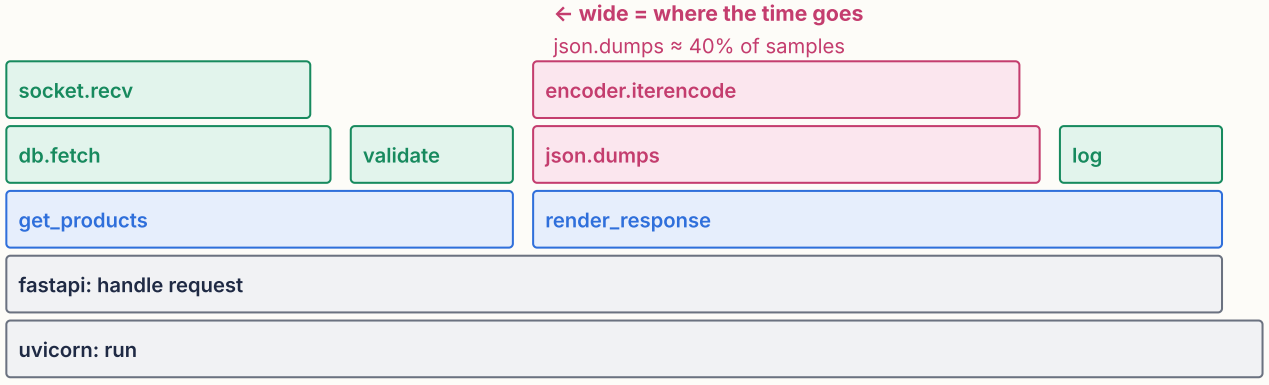


Figure 2. A flame graph (simplified): json.dumps under render_response is the widest leaf, about 40% of CPU.

4 Python tools



continuous profilers (Pyroscope, cloud profilers) sample all the time with tiny overhead

Figure 3. Four Python profilers and what each is best at.

`py-spy record -o flame.svg --pid 1234 --duration 60` attaches without code changes or restarts; `py-spy top --pid 1234` shows a live view. **Scalene** separates Python time from native time (is NumPy doing the work?), and **memray** tracks allocations to find leaks and spikes.

5 Async code and production

In asyncio, a profiler mostly shows the event loop, so look for coroutines that **block** the loop (lesson 38). py-spy's `--idle` flag includes idle time when you need it. In production, profile **one instance** for 30 to 60 seconds under real traffic. Continuous profilers (Pyroscope, Datadog, Google Cloud Profiler) sample all the time with tiny overhead.

WORKED EXAMPLE: A CPU-BOUND FASTAPI SERVICE

Run `py-spy record` on one worker for 60 seconds. The flame graph shows **40%** of the width under `json.dumps`, serialising large responses with the standard library. Switch to **orjson** (and FastAPI's `ORJSONResponse`), and CPU drops by a third while p99 improves. One measurement, one targeted fix.

6 Make it a habit

1. Reproduce with realistic load (next lesson).
2. Profile; find the widest frames.
3. Fix the biggest one.
4. Measure again: the profile changes.

And remember: CPU profiles don't show **waiting**. If latency is high but CPU is low, look at I/O, locks and downstream calls with distributed tracing instead. Profiles tell you what is busy; tracing tells you what is waiting.

COMMON MISTAKES

- Optimising code that isn't on the hot path.
- Running cProfile in production and blaming the slowdown on the app.
- Profiling with toy data, where the hot path is different.
- Reading flame graph x-position as time.

7 Key terms

Deterministic profiler: instruments every call (cProfile)

Sampling profiler: periodically samples stacks (py-spy)

Flame graph: stacked boxes, width = share of samples

Hot path: code where most time is spent

Continuous profiling: always-on low-overhead sampling

Tracing: timing requests across services, shows waits

8 Interview questions

Q1. How do you find where a Python service spends CPU time? BASIC
Attach a sampling profiler like py-spy to a running process under real load, generate a flame graph, and look for the widest frames.

Q2. Why sampling instead of cProfile in production? BASIC
Sampling has very low overhead and needs no code changes, while cProfile slows every call and distorts the results.

Q3. How do you read a flame graph? BASIC
Each box is a function on top of its caller; width is the share of samples, so the widest boxes, especially wide leaves, are where time goes; horizontal order is not time.

Q4. Latency is high but CPU is 20%. Will a profiler help? SCENARIO
Not much: the service is waiting. Use distributed tracing and metrics for I/O, locks, connection pools and downstream calls (and USE for resources).

Q5. Memory grows until pods are OOM-killed every day. What do you use? SCENARIO
A memory profiler such as memray or tracemalloc snapshots over time, to see which allocations grow; check caches without limits and objects kept alive by references.

Q6. Why can profiling an asyncio app be confusing? TRICKY
Most samples show the event loop waiting; the real issues are coroutines doing CPU work or blocking calls without awaits, which you find by looking for long synchronous frames inside tasks.

Next: SD-J-011 · Load Testing and Stress Testing backendroadmap.com